Adaptive Drone Swarm Behavior Simulation: Project Report

Anup Baigain, Madhusudan Pun, Toran Bahadur Niroula

Date: May 8, 2025

Abstract
This project presents the development of a 3D drone swarm simulation using Python, designed to replicate emergent behaviors inspired by natural swarm intelligence. The simulation incorporates key behavioral algorithms, including alignment, cohesion, separation, obstacle avoidance, and dynamic pattern formation, allowing drones to navigate complex environments collaboratively. Implemented using object-oriented programming and visualized with `matplotlib` in a 3D space, the system enables intuitive observation of swarm dynamics. The modular design supports expansion into more advanced features such as reinforcement learning and role-based tasking. This work serves as both a demonstration of core Python programming skills acquired through the Samsung Innovation Campus and a foundation for future research in multi-agent systems and autonomous aerial robotics.

1 Introduction

This project delivers a modular simulation environment in Python to model and visualize adaptive drone swarm behaviors inspired by biological systems. The simulation demonstrates emergent behaviors such as cohesion, alignment, separation, obstacle avoidance, and dynamic pattern formation, aiming to bridge the gap between theoretical swarm algorithms and practical implementation in complex environments.

2 Project Objectives and Scope

Core Objectives:

- Develop a modular simulation platform for drone swarms.
- Implement fundamental swarm behaviors: cohesion, alignment, and separation.
- Design adaptive collision avoidance.
- Enable dynamic pattern formation (circle, grid, triangle, etc.).
- Extend to 3D simulation and explore reinforcement learning for adaptability (as time allows).

Scope & Deliverables:

- Physics-based drone movement and collision detection.
- Swarm intelligence algorithms (cohesion, alignment, separation).
- Obstacle avoidance for both static (pillars) and dynamic (other drones) obstacles.
- User-selectable pattern formations.

- Visualization and interactive GUI.
- Technical documentation and performance analysis.

3 Software Architecture Overview

The system is organized into several key modules:

- **Physics Engine:** Simulates drone kinematics, velocity, and collision detection.
- **Swarm Behavior Module:** Implements Boids-inspired rules for cohesion, alignment, and separation, as well as PID-based goal seeking and obstacle avoidance.
- **Visualization System:** Uses Tkinter and Matplotlib for real-time 3D visualization and interactive controls.
- **Scenario Manager:** Configures start/end zones, obstacles (pillars), and pattern targets.

4 Implementation Details

4.1 Core Classes

- **Pillar:** Represents cylindrical obstacles in the simulation space, supporting collision checks.
- **Drone:** Models individual drones with position, velocity, and behavior logic, including:
  - Pattern Formation: Uses PD control to move drones into geometric patterns (circle, line, square, star, triangle, etc.).
  - Obstacle Avoidance: Combines PID control for goal seeking with Boids rules for inter-drone avoidance and obstacle repulsion.
- **Swarm:** Manages the collection of drones, scenario setup, pattern assignment, and simulation updates.
- **SimulatorGUI:** Provides an interactive GUI for mode switching (pattern formation/obstacle avoidance), parameter tuning, and real-time visualization.

4.2 Swarm Behaviors

- **Cohesion/Alignment/Separation:**
  In the drone swarm simulation, three fundamental behaviors govern the collective movement of the drones. Cohesion ensures that each drone steers toward the average position of its neighboring drones, maintaining swarm integrity and preventing fragmentation. Alignment causes drones to match velocities with nearby swarm members, resulting in synchronized and coordinated group movement. Separation maintains a minimum safe distance between drones through repulsion forces, effectively preventing collisions while allowing the swarm to navigate complex environments.

```
def calculate_avoidance_force(self, swarm, obstacles):
    avoidance_force = np.array([0.0, 0.0, 0.0])
    swarm_center = np.zeros(3)
    neighbor_count = 0

    # Boids implementation
    for other_drone in swarm.drones:
        dist = np.linalg.norm(self.position - other_drone.position)
        if dist < self.avoidance_radius:
            swarm_center += other_drone.position
            neighbor_count += 1
            avoidance_force += (self.position - other_drone.position)/dist**2
     # Separation

    if neighbor_count > 0:
        cohesion_force = (swarm_center/neighbor_count - self.position) * 0.1
    # Cohesion
         alignment_force = (np.mean([d.velocity for d in neighbors], axis=0) -
     self.velocity) * 0.1  # Alignment
```

- **Pattern Formation:** Drones can arrange themselves into user-selected formations, with smooth transitions between patterns. It uses matrix operations similar to Swarm-in-Blocks' formation system, For example:

```
def generate_pattern_targets(self, pattern_name):
    if pattern_name == "circle":
        radius = 7.5
        return np.array([[radius*np.cos(), radius*np.sin(), 0]
                         for  in np.linspace(0, 2, num_drones)])
    elif pattern_name == "square":
        return np.array([[x, y, 0] for x in [-5,0,5] for y in [-5,0,5]])
```

- **Obstacle Avoidance:** Drones detect and avoid pillars and other drones using repulsion forces and PID control.

```
# Pillar collision detection
dist_to_axis_sq = dx**2 + dy**2
is_within_height = position[2] <= self.height
return dist_to_axis_sq <= radius**2 and is_within_height

# Drone.update()
if obstacle.is_inside(drone.position):
    drone.eliminated = True
```

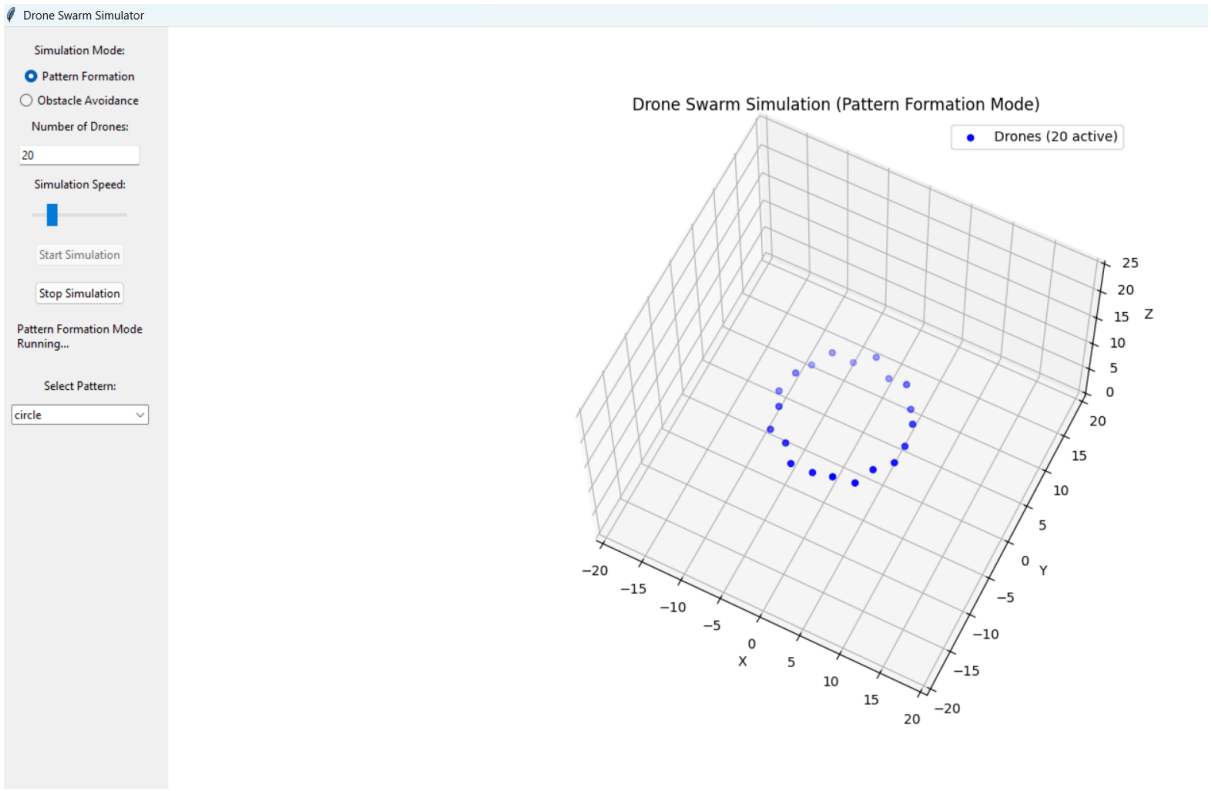


Figure 1: Circular pattern in 3D space

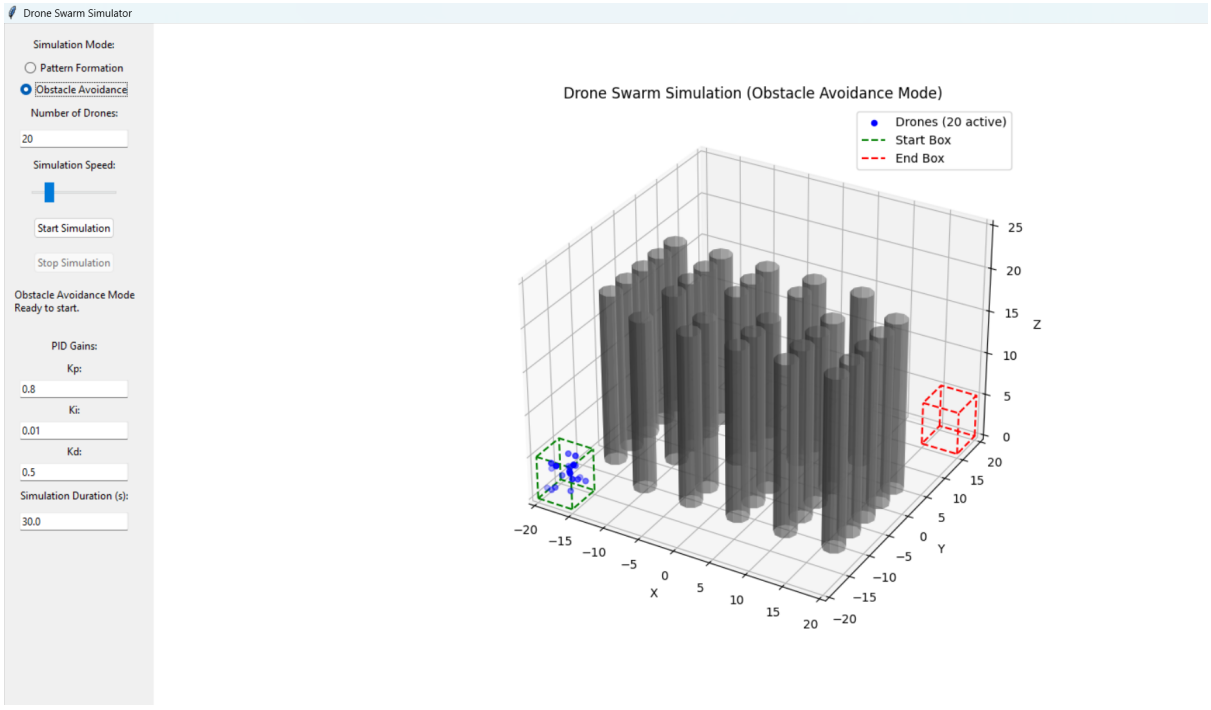


Figure 2: Pillar as obstacle in 3D space

4.3 Advanced Features

- **Hybrid Control System:**
  It combines PD control for pattern tracking with PID for goal seeking. It implements mode switching through GUI.

```
# Drone.update()
if mode == 'pattern_formation':
    force += calculate_pattern_force()
elif mode == 'obstacle_avoidance':
    force += calculate_pid_force() + calculate_avoidance_force()
```

- **Adaptive PID Controller:**
  Tunable gains (Kp=0.8, Ki=0.01, Kd=0.5) enable dynamic response adjustment. Includes anti-windup through velocity checks.

```
# PID force calculation
P_term = Kp * error
I_term = Ki * integral_error
D_term = Kd * derivative_error
```

- **3D Visualization:**
  Uses Matplotlib 3D projection for real-time visualization. Also, Renders obstacles as translucent cylinders.

```
# GUI.plot_swarm()
self.ax_swarm = self.figure.add_subplot(111, projection='3d')
ax.scatter(x,y,z, marker='o', c='b')
```

4.4 User Interaction

- Switch between ”Pattern Formation” and ”Obstacle Avoidance” modes.
- Select pattern type and number of drones.
- Adjust PID controller gains and simulation speed.
- Start, stop, and reset simulations with real-time feedback and results display.

5 Results and Features

- **Emergent Swarm Behavior:** The simulation demonstrates realistic flocking, pattern formation, and adaptive navigation through obstacle fields.
- **3D Visualization:** Real-time 3D plotting of drones, obstacles, and start/end zones.
- **Performance Metrics:** Tracks success rates, elimination counts, and average times for drones reaching goals.
- **Extensibility:** Modular codebase supports future integration of advanced features like leader election, adaptive formation switching, and reinforcement learning.

6 Conclusion

The project successfully meets its core objectives by providing a robust, interactive, and extensible platform for simulating and analyzing drone swarm behaviors. The implementation closely follows the proposed architecture and fulfills the planned deliverables, serving as a valuable tool for research and demonstration of multi-agent systems and swarm intelligence.

References

[1]

[2]